

tf.compat.v1.distribute.experimental.ParameterServerStrategy

Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/distribute/experimental/ParameterServerStrategy

An asynchronous multi-worker parameter server tf.distribute strategy.

Function Signatures

```
tf.compat.v1.distribute.experimental.ParameterServerStrategy(  
    cluster_resolver=None )  
call_for_each_replica(fn, ...)
```

Code Examples

```
tf.compat.v1.distribute.experimental.ParameterServerStrategy(  
    cluster_resolver=None  
)
```

```
tf.compat.v1.distribute.experimental.ParameterServerStrategy(  
    cluster_resolver=None  
)
```

```
strategy = tf.distribute.experimental.ParameterServerStrategy()
run_config = tf.estimator.RunConfig(
    experimental_distribute.train_distribute=strategy)
estimator = tf.estimator.Estimator(config=run_config)
tf.estimator.train_and_evaluate(estimator,...)
```

```
strategy = tf.distribute.experimental.ParameterServerStrategy()
run_config = tf.estimator.RunConfig(
    experimental_distribute.train_distribute=strategy)
estimator = tf.estimator.Estimator(config=run_config)
tf.estimator.train_and_evaluate(estimator,...)
```

```
os.environ['TF_CONFIG'] = json.dumps({
    'cluster': {
        'worker': ["localhost:12345", "localhost:23456"],
        'ps': ["localhost:34567"]
    },
    'task': {'type': 'worker', 'index': 0}
})
# This implicitly uses TF_CONFIG for the cluster and current task
info.
strategy = tf.distribute.experimental.MultiWorkerMirroredStrategy()
...
if strategy.cluster_resolver.task_type == 'worker':
    # Perform something that's only applicable on workers. Since we set
    this
    # as a worker above, this block will run on this particular
    instance.
elif strategy.cluster_resolver.task_type == 'ps':
    # Perform something that's only applicable on parameter servers.
    Since we
    # set this as a worker above, this block will not run on this
    particular
    # instance.
```

```

os.environ['TF_CONFIG'] = json.dumps({
    'cluster': {
        'worker': ["localhost:12345", "localhost:23456"],
        'ps': ["localhost:34567"]
    },
    'task': {'type': 'worker', 'index': 0}
})
# This implicitly uses TF_CONFIG for the cluster and current task
info.
strategy = tf.distribute.experimental.MultiWorkerMirroredStrategy()
...
if strategy.cluster_resolver.task_type == 'worker':
    # Perform something that's only applicable on workers. Since we set
    this
    # as a worker above, this block will run on this particular
    instance.
elif strategy.cluster_resolver.task_type == 'ps':
    # Perform something that's only applicable on parameter servers.
    Since we
    # set this as a worker above, this block will not run on this
    particular
    # instance.

```

```

distribute_datasets_from_function(
    dataset_fn, options=None
)

```

```

distribute_datasets_from_function(
    dataset_fn, options=None
)

```

Documentation

An asynchronous multi-worker parameter server `tf.distribute` strategy.

Inherits From: `Strategy`

This strategy requires two roles: workers and parameter servers. Variables and updates to those variables will be assigned to parameter servers and other operations are assigned to workers.

When each worker has more than one GPU, operations will be replicated on all GPUs. Even though operations may be replicated, variables are not and each worker shares a common view for which parameter server a variable is assigned to.

By default it uses `TFConfigClusterResolver` to detect configurations for multi-worker training. This requires a `'TF_CONFIG'` environment variable and the `'TF_CONFIG'` must have a cluster spec.

This class assumes each worker is running the same code independently, but parameter servers are running a standard server. This means that while each worker will synchronously compute a single gradient update across all GPUs, updates between workers proceed asynchronously. Operations that occur only on the first replica (such as incrementing the global step), will occur on the first replica of every worker.

It is expected to call `call_for_each_replica(fn, ...)` for any operations which potentially can be replicated across replicas (i.e. multiple GPUs) even if there is only CPU or one GPU. When defining the `fn`, extra caution needs to be taken:

- 1) It is generally not recommended to open a device scope under the strategy's scope. A device scope (i.e. calling `tf.device`) will be merged with or override the device for operations but will not change the device for

variables.

2) It is also not recommended to open a colocation scope (i.e. calling `tf.compat.v1.colocate_with`) under the strategy's scope. For colocating variables, use `strategy.extended.colocate_vars_with` instead. Colocation of ops will possibly create device assignment conflicts.

For Example:

`## Attributes`

In general, when using a multi-worker `tf.distribute` strategy such as `tf.distribute.experimental.MultiWorkerMirroredStrategy` or `tf.distribute.TPUStrategy()`, there is a `tf.distribute.cluster_resolver.ClusterResolver` associated with the strategy used, and such an instance is returned by this property.

Strategies that intend to have an associated `tf.distribute.cluster_resolver.ClusterResolver` must set the relevant attribute, or override this property; otherwise, `None` is returned by default. Those strategies should also provide information regarding what is returned by this property.

Single-worker strategies usually do not have a `tf.distribute.cluster_resolver.ClusterResolver`, and in those cases this property will return `None`.

The `tf.distribute.cluster_resolver.ClusterResolver` may be useful when the user needs to access information such as the cluster spec, task type or task id. For example,

For more information, please see

tf.distribute.cluster_resolver.ClusterResolver's API docstring.

Methods

`## distribute_datasets_from_function`

[View source](#)

Distributes `tf.data.Dataset` instances created by calls to `dataset_fn`.

The argument `dataset_fn` that users pass in is an input function that has a `tf.distribute.InputContext` argument and returns a `tf.data.Dataset` instance. It is expected that the returned dataset from `dataset_fn` is already batched by per-replica batch size (i.e. global batch size divided by the number of replicas in sync) and sharded.

`tf.distribute.Strategy.distribute_datasets_from_function` does not batch or shard the `tf.data.Dataset` instance returned from the input function. `dataset_fn` will be called on the CPU device of each of the workers and each generates a dataset where every replica on that worker will dequeue one batch of inputs (i.e. if a worker has two replicas, two batches will be dequeued from the Dataset every step).

This method can be used for several purposes. First, it allows you to specify your own batching and sharding logic. (In contrast, `tf.distribute.experimental_distribute_dataset` does batching and sharding for you.) For example, where `experimental_distribute_dataset` is unable to shard the input files, this method might be used to manually shard the dataset (avoiding the slow fallback behavior in `experimental_distribute_dataset`). In cases where the dataset is infinite, this sharding can be done by creating dataset replicas

that differ only in their random seed.

The `dataset_fn` should take an `tf.distribute.InputContext` instance where information about batching and input replication can be accessed.

You can use `element_spec` property of the `tf.distribute.DistributedDataset` returned by this API to query the `tf.TypeSpec` of the elements returned by the iterator. This can be used to set the `input_signature` property of a `tf.function`. Follow `tf.distribute.DistributedDataset.element_spec` to see an example.

For a tutorial on more usage and properties of this method, refer to the tutorial on distributed input).

If you are interested in last partial batch handling, read this section.

experimental_distribute_dataset

[View source](#)

Creates `tf.distribute.DistributedDataset` from `tf.data.Dataset`.

The returned `tf.distribute.DistributedDataset` can be iterated over similar to regular datasets.

NOTE: The user cannot add any more transformations to a `tf.distribute.DistributedDataset`. You can only create an iterator or examine the `tf.TypeSpec` of the data generated by it. See API docs of `tf.distribute.DistributedDataset` to learn more.

The following is an example:

Three key actions happening under the hood of this method are batching, sharding, and prefetching.

In the code snippet above, `dataset` is batched by `global_batch_size`, and calling `experimental_distribute_dataset` on it rebatches `dataset` to a new batch size that is equal to the global batch size divided by the number of replicas in sync. We iterate through it using a `Pythonic for` loop. `x` is a `tf.distribute.DistributedValues` containing data for all replicas, and each replica gets data of the new batch size. `tf.distribute.Strategy.run` will take care of